

Mini Banking & Payments System

The Complete Solution, Walked Through

Python OOP + Exception Handling • Post-test review session

171 tests · 6 tasks · 1,604 lines

FIL Fresher Training

How this session runs

Every task gets the same four beats. The third one is where the marks were lost.

1

The idea

What the task was really testing, in one sentence.

2

The code

The reference solution, verbatim — only type hints trimmed where a line had to fit.

3

The trap

The version most people wrote, and why it fails.

4

The proof

The test that catches it.

Interrupt at any point. A question about why beats a note about what.

The system at a glance

Six modules. Each one only knows about the ones above it — that is the whole architecture.

<code>money.py</code>	Money	Decimal + currency. Operator overloading.	221 lines
<code>transaction.py</code>	Transaction	Immutable, hashable ledger record.	141 lines
<code>mixins.py</code>	JSONSerializable / Auditable	Two capabilities, multiple inheritance.	86 lines
<code>accounts.py</code>	Account → Savings / Checking	Abstract base, encapsulated state.	525 lines
<code>strategies.py</code>	Interest / Fee / Composite	Duck-typed month-end policies.	103 lines
<code>bank.py</code>	Bank	Factory, transfers, orchestration.	325 lines

Four rules decided most of the marks

None of these are Python trivia. All four are things a real banking codebase is audited for.

1

No float, ever

$0.1 + 0.2 \neq 0.3$. A ledger that drifts by a paisa per transaction is a ledger nobody trusts.

2

No bare except

``except:`` swallows the `KeyboardInterrupt` and the typo in your own code alongside the error you meant to handle.

3

No public mutable state

If ``account.balance = 1_000_000`` works, every rule the class enforces is decoration.

4

Chain your exceptions

``raise X from exc`` keeps the original traceback. Without it, you lose the only evidence of what actually broke.

TASK 1

Money

`minibank/money.py`

A value object that behaves like a number, refuses to lose a paisa, and refuses to add rupees to dollars.

What it tests

- Operator overloading
- Immutability
- Hash / equality contract
- Decimal discipline

Why not just use float?

Run this in any Python REPL. It is not a Python bug — it is binary floating point doing exactly what IEEE 754 says.

python

```
>>> 0.1 + 0.2
0.30000000000000004

>>> total = 0.0
>>> for _ in range(10):
...     total += 0.1
>>> total == 1.0
False
>>> total
0.9999999999999999
```

the same thing with Decimal

```
>>> from decimal import Decimal
>>> Decimal('0.1') + Decimal('0.2')
Decimal('0.3')

>>> total = Money.zero()
>>> for _ in range(10):
...     total = total + Money('0.10')
>>> total == Money('1.00')
True
```

0.1 has no exact binary representation, the same way $1/3$ has no exact decimal one. Every addition compounds the error.

A bank with 10 million transactions a day and a rounding error of one hundredth of a paisa per transaction is out by roughly ₹365 a year — and cannot tell you which transactions caused it. That is the audit failure, not the money.

Money: the whole declaration

TASK 1 · money.py

Three decorators do the work that would otherwise be forty lines of boilerplate.

```
@total_ordering
@dataclass(frozen=True, order=False)
class Money:
    amount: Decimal
    currency: str = DEFAULT_CURRENCY
```

@dataclass(frozen=True)

Generates `__init__` and blocks attribute assignment. Immutability for free.

order=False

Stops the dataclass generating comparisons that would ignore currency.

@total_ordering

Derives three comparison dunder methods from `__lt__` and `__eq__`.

Reach for the standard library before you hand-write a protocol. It is shorter and it is already correct.

Normalising in a frozen class

Frozen means you cannot assign. It does not mean you cannot clean up inputs — you just need the right door.

money.py — `__post_init__`

```
def __post_init__(self) -> None:
    try:
        raw = str(self.amount) if isinstance(self.amount, float) else self.amount
        amount = Decimal(raw).quantize(PLACES, rounding=ROUND_HALF_UP)
    except (InvalidOperation, TypeError, ValueError) as exc:
        raise InvalidAmountError(self.amount, 'unparseable amount') from exc

    currency = str(self.currency).upper()
    if re.fullmatch(r'[A-Z]{3}', currency) is None:
        raise InvalidAmountError(self.currency, 'currency must be three letters')

    object.__setattr__(self, 'amount', amount)      # the door through frozen
    object.__setattr__(self, 'currency', currency)
```

object.__setattr__ bypasses the frozen guard. Legitimate inside `__post_init__` — nowhere else.

The float trapdoor inside Decimal

Using Decimal is not enough. How you feed it decides whether it helps you.

✗ Decimal(a float)

```
>>> Decimal(0.1)
Decimal('0.1000000000000000055511
151231257827021181583404541015625')

# Money(0.1) built this way starts
# life already wrong. quantize()
# then hides it at 2dp.
```

The damage is done before quantize runs. You have laundered a float error into a Decimal.

✓ Decimal(str(the float))

```
>>> Decimal(str(0.1))
Decimal('0.1')

raw = (str(self.amount)
        if isinstance(self.amount, float)
        else self.amount)
amount = Decimal(raw).quantize(...)
```

repr() of a float gives the shortest string that round-trips — which is the number the human meant.

Never hand a float straight to Decimal. Convert through str, or refuse floats outright.

Rounding is a business decision

Python's default rounding is not the one a customer expects. The spec named `ROUND_HALF_UP` for a reason.

input	ROUND_HALF_UP (correct)	ROUND_HALF_EVEN (default)	
Money('100.005')	100.01	100.00	half-way goes up
Money('100.015')	100.02	100.02	same either way
Money('100.025')	100.03	100.02	banker's rounds to even
Money('100.555')	100.56	100.56	same either way

`quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)` – say it explicitly; the default is `ROUND_HALF_EVEN`.

Banker's rounding is defensible and it is still wrong here, because the spec asked for something else.

Return NotImplemented — do not raise

The single most common mistake in the whole test. Both versions look like they work.

✗ Raising TypeError yourself

```
def __add__(self, other):  
    if not isinstance(other, Money):  
        raise TypeError('need Money')  
    ...
```

You just vetoed every type that might have known how to add itself to a Money — a Portfolio, a Mock, a numpy scalar. Python never gets to ask them.

✓ Returning NotImplemented

```
def __add__(self, other):  
    if not isinstance(other, Money):  
        return NotImplemented  
    self._check_currency(other)  
    return Money(self.amount + other.amount,  
                self.currency)
```

Python then tries `other.__radd__(self)`. If that also declines, Python raises the `TypeError` — with a better message than yours.

`Money('100') + 5` still raises `TypeError` either way. The difference is who decides — you, or the language. Only one of those answers stays correct when someone writes a class you have never seen.

One comparison, not four

TASK 1 · money.py

Four hand-written comparisons are four chances to get a sign backwards — and the bug shows up in sorted() output six months later.

✗ Writing them all by hand

```
def __lt__(self, o): ...
def __le__(self, o): ...
def __gt__(self, o): ...
def __ge__(self, o): ...

# four currency checks to keep in
# sync, four places to typo < as >
```

✓ @total_ordering + __lt__

```
@total_ordering
class Money:
    def __lt__(self, other):
        if not isinstance(other, Money):
            return NotImplemented
        self._check_currency(other)
        return self.amount < other.amount
```

`total_ordering` needs `__eq__` and exactly one of `__lt__` / `__le__` / `__gt__` / `__ge__`. It fills in the rest at class-creation time.

Write the one definition that carries the meaning. Derive the rest.

`__eq__` and `__hash__` travel together

Define one without the other and you get a type that silently misbehaves in sets and dicts.

the contract, in code

```
def __eq__(self, other):
    if not isinstance(other, Money):
        return False # never raises
    return (self.amount == other.amount
            and self.currency == other.currency)

def __hash__(self):
    return hash((self.amount, self.currency))
```

`a == b` must imply `hash(a) == hash(b)`

Otherwise `{Money('10')}` and `Money('10.00')` land in different buckets and your set has duplicates.

Defining `__eq__` sets `__hash__` to `None`

A class with `__eq__` and no `__hash__` is unhashable. `frozen=True` restores it — unless you override `__eq__`, as we do here.

Comparing to a non-Money returns False

`Money('0') == 0` is False, not an exception. `__eq__` must answer every question.

Equal objects must hash equally. Unequal objects may collide. Breaking the first rule breaks every dict you put them in.

TASK 2

Transaction

`minibank/transaction.py`

A record of something that already happened. It is frozen because the past does not change, and hashable because replayed batches need de-duplicating.

What it tests

- Immutability by default
- Enum over strings
- Identity vs value equality
- Timezone discipline

Where does the minus sign live?

Two models for the same ledger. One of them makes an invalid state unrepresentable.

✗ Signed amount, no type

```
Transaction('T1', Money('-500'))
Transaction('T2', Money('500'))

# What is a -500 DEPOSIT?
# What is a +500 FEE?
# Both are representable. Both
# are nonsense. Now every reader
# must defend against them.
```

✓ Positive amount + a type

```
amount: always positive Money
type:   a TransactionType member

@property
def signed_amount(self):
    return (self.amount
            if self.type.is_credit
            else -self.amount)
```

Make illegal states unrepresentable. The constructor rejects a negative amount, so no reader downstream has to check.

Enum, not string literals

The enum was given to you. Here is what it bought.

transaction.py — given

```
class TransactionType(enum.Enum):
    DEPOSIT = 'DEPOSIT'; WITHDRAWAL = 'WITHDRAWAL'
    TRANSFER_IN = 'TRANSFER_IN'; TRANSFER_OUT = 'TRANSFER_OUT'
    INTEREST = 'INTEREST'; FEE = 'FEE'

    @property
    def is_credit(self) -> bool:
        # the rule lives WITH the data
        return self in {TransactionType.DEPOSIT, TransactionType.TRANSFER_IN,
                        TransactionType.INTEREST}
```

Typos fail loudly

'DEPOSITT' is a silent bug.
TransactionType.DEPOSITT is an AttributeError at import.

Behaviour has a home

is_credit belongs to the type itself, so no if/elif chain gets copied into four modules.

The set is closed

You can enumerate every kind of entry. A string field has infinite possible values.

Hash on identity, not on contents

TASK 2 · transaction.py

Money hashes on its value. Transaction hashes on its id. The difference is not an inconsistency — it is the point.

transaction.py

```
def __hash__(self):  
    return hash(self.txn_id)  
  
def __eq__(self, other):  
    if not isinstance(other, Transaction):  
        return NotImplemented  
    return self.txn_id == other.txn_id
```

Value object — Money

Two ₹10 notes are interchangeable. Identity is the value. Hash everything.

Entity — Transaction

Two records with the same id are the same event, even if a description was edited. Hash the id.

This is what lets `set(replayed_batch)` drop duplicates: same id, same event, one entry.

Naive datetimes are a bug

datetime.now() has no timezone. It is a timestamp whose meaning depends on which server wrote it.

✗ `datetime.now()`

```
>>> datetime.now()
datetime(2025, 9, 22, 11, 3, 15)
>>> _.tzinfo
None

# Mumbai server: 11:03 IST
# London replica: 11:03 BST
# Same string. Different instant.
```

✓ `datetime.now(timezone.utc)`

```
def _utc_now() -> datetime:
    return datetime.now(timezone.utc)

timestamp: datetime = field(
    default_factory=_utc_now)

# 2025-09-22T05:33:15+00:00
# unambiguous anywhere on earth
```

default_factory, not a default value — a bare default would freeze one timestamp at import time for every transaction.

TASK 3

Mixins

`minibank/mixins.py`

Two small classes that are never instantiated on their own. Together they are the only multiple inheritance in the codebase — and the only place it earns its keep.

What it tests

- Cooperative `super()`
- Method resolution order
- Exception chaining
- Single responsibility

What makes a class a mixin

A mixin does

- Adds exactly one capability
- Never instantiated on its own
- Declares no state that clashes with the host
- Calls `super().__init__(*args, **kwargs)`
- Can be dropped onto any unrelated class

A mixin does not

- Is not a base class you subclass for identity
- Does not know its host's fields
- Does not sit alone in the MRO tail
- Is not a namespace for loose helper functions

Our two: `JSONSerializableMixin` gives a class `to_json`. `AuditableMixin` gives it an event trail. Neither knows what an `Account` is — both work on any class you drop them onto.

The test proves it: the suite mixes both into a throwaway `Widget` class that has nothing to do with banking.

A hook the host must fill

The mixin cannot know your fields. So it defines the shape of the answer and makes the host supply it.

`mixins.py` — `JSONSerializableMixin`

```
def to_dict(self) -> Dict[str, Any]:
    raise NotImplementedError(f'{type(self).__name__} must implement to_dict()')

def to_json(self, *, indent: int | None = None) -> str:
    return json.dumps(self.to_dict(), indent=indent, default=str)

@classmethod
def from_json(cls, payload: str) -> Dict[str, Any]:
    try:
        return json.loads(payload)
    except json.JSONDecodeError as exc:
        raise BankError('invalid JSON payload') from exc
```

`type(self).__name__`

The error names the concrete class, so the message reads 'SavingsAccount must implement to_dict()' — not the mixin's own name.

`default=str`

json cannot encode Decimal or datetime. One argument makes the whole tree serialisable instead of writing a custom JSONEncoder.

raise ... from exc

TASK 3 · mixins.py

Three ways to re-raise. Only one of them keeps the evidence.

✗ except: pass

```
try:
    return json.loads(payload)
except json.JSONDecodeError:
    return {}
```

The corrupt payload is now an empty dict. The bug surfaces three layers away as a `KeyError`.

✗ raise without from

```
except json.JSONDecodeError:
    raise BankError('invalid JSON')
```

Python prints 'During handling of the above exception, another occurred' — context kept, but the causal link is guesswork.

✓ raise ... from exc

```
except json.JSONDecodeError as exc:
    raise BankError(
        'invalid JSON payload') from exc
```

Sets `__cause__`. The traceback reads 'The above exception was the direct cause' — a stated fact, not an accident of timing.

The test asserts `isinstance(exc.__cause__, json.JSONDecodeError)`. Chaining is not style — it is checked.

The one line that breaks everything

TASK 3 · mixins.py

AuditableMixin.__init__ has four lines. Leave one out and the failure lands somewhere else entirely.

mixins.py — AuditableMixin

```
def __init__(self, *args: Any, **kwargs: Any) -> None:
    self._audit_log: List[str] = []
    super().__init__(*args, **kwargs)      # <-- hand control along the MRO
```

✗ Forgetting super()

```
def __init__(self, *args, **kwargs):
    self._audit_log = []
    # chain stops here
```

Every class after AuditableMixin in the MRO is skipped. No error. No warning. Attributes those classes would have set simply do not exist, and you find out via `AttributeError` much later.

✓ Passing *args, **kwargs through

```
def __init__(self, *args, **kwargs):
    self._audit_log = []
    super().__init__(*args, **kwargs)
```

The mixin does not know or care who comes next, or what arguments they need. It takes its slice and forwards the rest untouched. That is what 'cooperative' means.

Reading the MRO

super() does not mean 'my parent class'. It means 'the next class in this object's MRO' — and the MRO depends on the instance, not the file.

```
class Account(JSONSerializableMixin, AuditableMixin, ABC): ...

>>> [c.__name__ for c in SavingsAccount.__mro__]
['SavingsAccount', 'InterestBearingAccount', 'Account',
 'JSONSerializableMixin', 'AuditableMixin', 'ABC', 'object']
```

SavingsAccount ›

InterestBearing ›

Account ›

JSONSerializable ›

Auditable ›

ABC ›

object

`Account.__init__` calls `super().__init__()` — which lands on `JSONSerializableMixin` (no `__init__`, so Python walks on) and then on `AuditableMixin`, which creates the audit log. Break the chain at the highlighted box and the log never exists.

C3 linearisation, left to right, depth-first, never repeating. Print `__mro__` when in doubt — it is not worth reasoning about from memory.

Order inside Account.__init__

TASK 3 · accounts.py

The audit log must exist before anything tries to write to it. That constrains the order of four statements.

accounts.py — Account.__init__ (tail)

```
self._account_id = account_id
self._owner = owner.strip()
self._currency = zero.currency
self._balance = zero
self._ledger: List[Transaction] = []
self._is_closed = False

super().__init__()          # 1. mixin creates _audit_log
self.audit(                 # 2. now this is safe
    f'opened {self.account_type} account for {self.owner}')

if opening_balance:         # 3. and only now, money moves
    self.deposit(opening_balance, 'opening balance')
```

Own state first

self.account_type is an abstract property that reads subclass state — it must already be set.

super() next

Creates _audit_log by walking the MRO.

Audit third

The first event in the trail: the account opening.

Deposit last

Goes through the public API, so the opening balance gets a real ledger entry.

TASK 4

Accounts

`minibank/accounts.py`

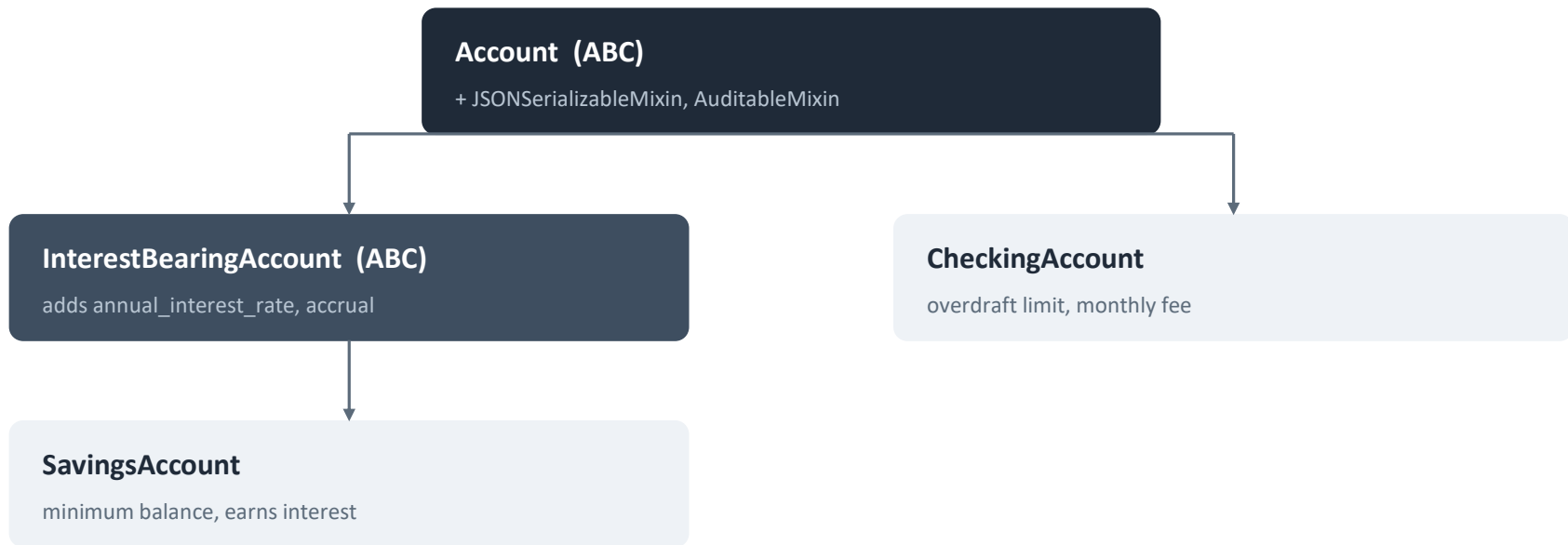
525 lines and the heart of the exercise. An abstract base that writes every algorithm once, and two subclasses that supply only what differs.

What it tests

- Encapsulation
- Abstract contracts
- Polymorphism
- Template method

The hierarchy, and its middle layer

TASK 4 · accounts.py



Why not put interest on Account? Because then **CheckingAccount** would inherit `accrue_monthly_interest()` and have to refuse it at runtime. The test asserts `hasattr(checking, 'accrue_monthly_interest')` is `False` — the absence is the design.

Encapsulation that actually holds

A leading underscore is a convention. A read-only property is enforcement.

✗ Public attribute

```
self.balance = Money.zero(currency)

>>> acct.balance = Money('1000000')
>>> acct.balance
Money('1000000.00', 'INR')
```

Every rule the class enforces is now optional.

✓ Private state + property

```
self._balance = zero

@property
def balance(self) -> Money:
    return self._balance    # no setter
```

acct.balance = ... now raises AttributeError.

and the defensive copy — accounts.py

```
@property
def ledger(self) -> tuple:
    return tuple(self._ledger)    # a caller cannot append to the real list
```

Return a tuple, not the list. Handing out your internal list is handing out a setter you did not mean to write.

Three abstract members, one contract

Everything else — `deposit`, `withdraw`, `close`, `__len__`, `to_dict` — is written once in the base class.

member	SavingsAccount	CheckingAccount
<code>account_type</code>	<code>'SAVINGS'</code>	<code>'CHECKING'</code>
<code>withdrawable_balance</code>	<code>balance - minimum_balance</code> <code>(floored at zero)</code>	<code>balance + overdraft_limit</code>
<code>_insufficient_funds_error()</code>	<code>InsufficientFundsError</code>	<code>OverdraftLimitExceededError</code> <code>(carries .limit)</code>

Name the difference, not the behaviour. Two accounts differ by one number and one exception type — so that is all the subclasses say.

Write withdraw once

The template method pattern: the algorithm lives in the base class, the varying step is abstract.

✗ isinstance chain in the base

```
def withdraw(self, amount, desc=''):
    if isinstance(self, SavingsAccount):
        limit = self.balance - self.minimum
    elif isinstance(self, CheckingAccount):
        limit = self.balance + self.overdraft
    ...
```

The base class now imports its own subclasses. Adding a third account type means editing this method — and the next one, and the next.

✓ One abstract property

```
def _debit(self, txn_type, amount, desc):
    self._validate(amount)
    if amount > self.withdrawable_balance:
        self.audit(f'{txn_type.value.lower()} '
                  'rejected: insufficient funds')
        raise self._insufficient_funds_error(amount)
    return self._apply(txn_type, amount, desc)
```

A new account type supplies two members and inherits every algorithm, unchanged.

The test asserts 'withdraw' not in vars(SavingsAccount). If you overrode it, you solved the wrong problem correctly.

One debit path, two entry points

withdraw and transfer_out follow identical rules and differ only in the label on the ledger entry.

accounts.py

```
def withdraw(self, amount, description=''):
    return self._debit(TransactionType.WITHDRAWAL, amount, description)

def transfer_out(self, amount, description=''):
    return self._debit(TransactionType.TRANSFER_OUT, amount, description)

def _debit(self, txn_type, amount, description):
    """Validate and apply a debit that respects the account's limit."""
    self._validate(amount)
    if amount > self.withdrawable_balance:
        self.audit(f'{txn_type.value.lower()} rejected: insufficient funds')
        raise self._insufficient_funds_error(amount)
    return self._apply(txn_type, amount, description)
```

Copy-pasting `_debit`'s body into `transfer_out` would pass every test today. It fails the day someone changes the validation order in one copy and not the other — and that failure is a production incident, not a test failure.

Validation order is part of the contract

TASK 4 · accounts.py

Four checks. The order decides which error a caller sees when two things are wrong at once.

accounts.py — `_validate`

```
def _validate(self, amount: Money) -> None:
    if self.is_closed:
        raise AccountClosedError(self.account_id)
    if not isinstance(amount, Money):
        raise InvalidAmountError(amount, 'amount must be Money')
    if amount.currency != self.currency:
        raise CurrencyMismatchError(self.currency, amount.currency)
    if not amount.is_positive:
        raise InvalidAmountError(amount, 'amount must be positive')
```

1. Closed?

Cheapest check, and it makes every later question moot.

2. Is it Money?

Must precede any attribute access — `amount.currency` on an `int` is an `AttributeError`, not a domain error.

3. Right currency?

A structural question about the value.

4. Positive?

A business rule, last. Money itself allows negatives; the account does not.

Cheap and structural checks before expensive and semantic ones — and never touch an attribute before you know the type.

One place writes the balance

TASK 4 · accounts.py

Six operations change money. All six go through one method — so there is one place to audit, log, or put a breakpoint.

accounts.py — _apply

```
def _apply(self, txn_type, amount, description='') -> Transaction:
    self._balance = (
        self._balance + amount if txn_type.is_credit
        else self._balance - amount
    )
    transaction = Transaction(
        self._next_txn_id(), txn_type, amount,
        self._balance, description
    )
    self._ledger.append(transaction)
    return transaction
```

Everything funnels here

deposit() →

withdraw() →

transfer_in() →

transfer_out() →

accrue_monthly_interest() →

opening_balance() →

_apply

is_credit decides the sign, so **_apply** has no if/elif over six transaction types. The enum already knows.

A refused withdrawal changes nothing

Check before you mutate. The order of those two steps is the difference between a rejection and a corruption.

✗ Mutate, then check

```
self._balance -= amount
if self._balance < self.minimum:
    self._balance += amount # 'undo'
    raise InsufficientFundsError(...)
```

It happens to work here. It stops working the moment anything between the two lines can fail, or another thread reads the balance.

✓ Check, then mutate

```
self._validate(amount)
if amount > self.withdrawable_balance:
    self.audit('... rejected ...')
    raise self._insufficient_funds_error(amount)
return self._apply(txn_type, amount, desc)
```

The balance is never briefly wrong. There is no window in which an observer sees an invalid state.

The rejection still leaves a trace — in the audit trail, not the ledger. The ledger records money that moved; the audit trail records what happened. A declined withdrawal is an event worth keeping and not a transaction.

Interest, and the zero-noise rule

TASK 4 · accounts.py

accounts.py — InterestBearingAccount

```
def monthly_interest(self) -> Money:
    if not self.balance.is_positive:
        return Money.zero(self.currency)      # never pay on an overdraft
    return self.balance * self.annual_interest_rate * (Decimal(1)/Decimal(12))

def accrue_monthly_interest(self) -> Optional[Transaction]:
    if self.is_closed:
        return None
    interest = self.monthly_interest()
    if not interest:                          # Money.__bool__ — zero is falsey
        return None
    return self._apply(TransactionType.INTEREST, interest, 'monthly interest')
```

₹12,000 at 12% p.a.

$12000 \times 0.12 \div 12 = ₹120.00$
new balance ₹12,120.00

Returns Optional

None means 'nothing happened'. A zero-rupee INTEREST row is noise in every statement forever.

Never on an overdraft

A negative balance would produce negative interest — a credit that silently deepens the debt.

if not interest

Money.__bool__ returns False at exactly zero, so this reads as English and stays exact.

Extend the parent's dict, do not rebuild it

TASK 4 · accounts.py

✗ Restating every key

```
def to_dict(self):
    return {
        'account_id': self.account_id,
        'account_type': self.account_type,
        'owner': self.owner,
        # ... six more, copied ...
        'minimum_balance': ...,
    }
```

Add a field to Account and you must remember to add it here, and in CheckingAccount, and in whatever ships next quarter.

✓ super().to_dict() then update

```
def to_dict(self) -> Dict[str, Any]:
    result = super().to_dict()
    result.update({
        'minimum_balance':
            str(self.minimum_balance.amount),
        'interest_rate':
            str(self.annual_interest_rate),
    })
    return result
```

The subclass states only what it adds. New base fields appear everywhere automatically.

Every override should be readable as a diff against the thing it overrides.

TASK 5

Strategies

`minibank/strategies.py`

103 lines, no base class, no interface declaration, no registration. Three classes that share nothing but a shape — and that is deliberate.

What it tests

- Duck typing
- Composition over inheritance
- Narrow exception handling
- Open/closed principle

No base class. On purpose.

A strategy is anything with a name and a callable apply. Nothing inherits from anything.

the entire 'interface'

```
class SomeStrategy:
    name: str
    def apply(self, account) -> list[Transaction]:
        ...

# no ABC, no Protocol, no registration,
# no import of anything in minibank
```

and the Bank validates shape, not type

```
apply = getattr(strategy, 'apply', None)
if not callable(apply):
    raise TypeError(
        'strategy is missing a callable apply method')
if not hasattr(strategy, 'name'):
    raise TypeError('strategy is missing a name')
```

Why this matters here

A team in another repo can write a month-end policy without importing your package or asking your permission. The test proves it: `class CountingStrategy` in test_bank.py inherits from nothing at all and the Bank accepts it. An ABC would have required them to depend on you first.

The one isinstance that belongs

Having just argued against type checks, here is the case where a type check is the correct answer.

strategies.py — InterestStrategy

```
def apply(self, account: Account) -> List[Transaction]:
    if account.is_closed or not isinstance(account, InterestBearingAccount):
        return []
    transaction = account accrue_monthly_interest()
    return [] if transaction is None else [transaction]
```

Why not hasattr?

`hasattr(account, 'accrue_monthly_interest')` would also pass for a class that happens to define a method with that name and completely different semantics. The capability is defined by the abstract class, so the abstract class is what you ask about.

The rule of thumb

Check the type when the type IS the contract (an ABC you own). Check the shape when you are accepting something from outside your world. Never check a concrete class — that is the `isinstance` chain again.

Charge the fee through the front door

A fee must not be able to break a rule the account guarantees — so it goes through withdraw, not _apply.

strategies.py — MaintenanceFeeStrategy

```
def apply(self, account: Account) -> List[Transaction]:
    if account.is_closed:
        return []
    fee = account.monthly_fee()
    if not fee.is_positive:
        # savings has no fee — no-op, no entry
        return []
    try:
        return [account.withdraw(fee, 'monthly maintenance fee')]
    except InsufficientFundsError:
        # THIS error, not any error
        account.audit('maintenance fee rejected: insufficient funds')
        return []
```

The test that makes the point: a ₹150 fee on a checking account holding ₹100 with a ₹40 overdraft limit must be REFUSED. Going through _apply would have forced it through and left the account ₹50 past a limit the class promises to enforce.

What a bare except actually catches

Three widths of net. Two of them catch things you never meant to touch.

✗ `except:`

```
try:
    account.withdraw(fee)
except:
    pass
```

Catches `KeyboardInterrupt`, `SystemExit`, `MemoryError` — and the `AttributeError` from your own typo. Ctrl-C stops working.

✗ `except Exception:`

```
try:
    account.withdraw(fee)
except Exception:
    pass
```

Spares Ctrl-C, still swallows the `TypeError` from a bug you introduced last week. The fee silently stops being charged and nobody finds out.

✓ `except InsufficientFundsError:`

```
try:
    return [account.withdraw(fee, '...')]
except InsufficientFundsError:
    account.audit('fee rejected: ...')
    return []
```

Catches the one outcome you predicted and handled. Everything else — including your own bugs — propagates and gets found.

Catch the narrowest exception you can actually do something about. Everything else is someone else's problem, and hiding it is not help.

Composition beats inheritance

Combining two policies needs a list, not a class hierarchy.

TASK 5 · strategies.py

strategies.py — CompositeStrategy

```
def __init__(self, *strategies: object) -> None:
    for strategy in strategies:
        if not callable(getattr(strategy, 'apply', None)):
            raise TypeError(
                'strategy must provide a callable apply method')
    self._strategies = tuple(strategies)

@property
def name(self) -> str:
    return '+'.join(str(getattr(strategy, 'name'))
                    for strategy in self._strategies)

def apply(self, account: Account) -> List[Transaction]:
    transactions: List[Transaction] = []
    for strategy in self._strategies:
        transactions.extend(strategy.apply(account))
    return transactions
```

The inheritance version

```
class InterestAndFeeStrategy(InterestStrategy,
                             MaintenanceFeeStrategy)
```

Now write the other five combinations. Then add a third policy.

The composition version

```
CompositeStrategy(
    InterestStrategy(),
    MaintenanceFeeStrategy())
```

n policies, any combination, zero new classes. And composites nest inside composites.

TASK 6

Bank

`minibank/bank.py`

The orchestrator. It creates accounts it knows nothing about, moves money atomically between them, and never once touches their private state.

What it tests

- Factory pattern
- Open/closed principle
- Compensating transactions
- Failure isolation

A registry, not an if/elif chain

Open for extension, closed for modification — the concrete, testable version of that slogan.

✗ Branching on the kind

```
if kind == 'SAVINGS':
    return SavingsAccount(...)
elif kind == 'CHECKING':
    return CheckingAccount(...)
else:
    raise UnknownAccountTypeError(kind)
```

Every new account type means editing this method — in a file that is already tested and already in production.

✓ A registry the factory reads

```
ACCOUNT_TYPES: Dict[str, Type[Account]] = {}

@classmethod
def register_account_type(cls, kind, account_cls):
    if not isinstance(account_cls, type) or not issubclass(
        account_cls, Account):
        raise TypeError(
            'account_cls must be an Account subclass')
    cls.ACCOUNT_TYPES[str(kind).upper()] = account_cls
```

A new type registers itself. `open_account` is never edited again.

The test proves it holds: it defines `ZeroFeeSavings` inside the test function, registers it, and opens one — without a single line of `bank.py` changing. If your factory needed editing to pass that test, it was not a factory.

@classmethod vs @staticmethod

TASK 6 · bank.py

Both appear in bank.py, one line apart. Choosing between them is not a style question.

@classmethod — needs the class

```
@classmethod
def register_account_type(cls, kind, account_cls):
    if not isinstance(account_cls, type) or not issubclass(
        account_cls, Account):
        raise TypeError(
            'account_cls must be an Account subclass')
```

register_account_type

@staticmethod — needs nothing

```
@staticmethod
def validate_account_id(account_id: str) -> str:
    if not isinstance(account_id, str) or re.fullmatch(
        r'[A-Za-z]{2}\d{4}', account_id
    ) is None:
        raise ValueError(f'invalid account id: {account_id!r}')
```

validate_account_id

Touches cls

Yes — mutates the shared registry

No

Touches self

No

No

Subclass-aware

Yes — cls is the actual subclass

Not applicable

Why not a plain function?

It is the class's own registry

Discoverability: it belongs to Bank's vocabulary

If it reads self, it is a method. If it reads cls, classmethod. If it reads neither, staticmethod — and ask once whether it belongs in the class at all.

The compensating transaction

TASK 6 · bank.py

There is no database and no rollback. Atomicity has to be written by hand — this is what that looks like.

bank.py — Bank.transfer

```
if str(from_id).upper() == str(to_id).upper():
    raise TransferError('cannot transfer to the same account')
source = self.get_account(from_id)
destination = self.get_account(to_id)
if source.is_closed:
    raise AccountClosedError(source.account_id)
if destination.is_closed:
    raise AccountClosedError(destination.account_id)
if source.currency != destination.currency:
    raise CurrencyMismatchError(source.currency, destination.currency)

# may raise — if it does, nothing has changed anywhere
debit = source.transfer_out(amount, description)
try:
    credit = destination.transfer_in(amount, description)
except BankError as exc:
    source.transfer_in(amount, f'rollback: {description}'.rstrip())
    raise TransferError(
        'transfer credit failed; debit rolled back') from exc
return debit, credit
```

All checks first

Same account, closed, currency — every cheap refusal happens before any money moves.

Debit can fail safely

If the source cannot afford it, transfer_out raises and no state changed anywhere.

The dangerous window

Between the debit and the credit, the money exists nowhere. This is the only risky moment.

Compensate, do not undo

The reversal is a NEW transfer_in, visible in the ledger. You cannot erase history — you can only add to it.

Why the rollback is visible

TASK 6 · bank.py

Three ledger entries for a transfer that never completed. That is correct, not messy.

id	type	amount	balance after	description
T000014	TRANSFER_OUT	- 1,000.00	9,000.00	rent
T000015	TRANSFER_IN	+ 1,000.00	10,000.00	rollback: rent

An auditor can see what happened

A silent undo leaves a gap in the transaction ids and no explanation. An auditor finding a gap assumes the worst, correctly.

__cause__ keeps the real reason

`raise TransferError(...)` from `exc` means the traceback names the original failure, not just 'transfer failed'.

The invariant that is tested

`total_assets()` is unchanged by a completed transfer AND by a rolled-back one. Money is conserved either way.

One bad account is not a bad month

TASK 6 · bank.py

Batch jobs fail partially. The question is only whether you planned for it.

bank.py — Bank.monthly_process

```
results: Dict[str, List[Transaction]] = {}
for account in self._accounts.values():
    try:
        results[account.account_id] = apply(account)
    except BankError as exc:
        account.audit(f"{getattr(strategy, 'name')} failed: {exc}")
        results[account.account_id] = []
return results
```

The loop keeps going

Account 400 failing must not deny interest to accounts 401 through 10,000.

BankError, not Exception

A domain failure is isolated. A bug in your own code still crashes the run — loudly, which is what you want.

Every account appears

Even a failed one gets a key with an empty list, so the caller can reconcile counts.

The failure is recorded

In that account's own audit trail, where whoever investigates will actually look.

The exception hierarchy was the design brief

This module was given to you complete. Reading it told you most of what the system had to do.

```
BankError
├─ CurrencyMismatchError      .left .right
├─ InvalidAmountError (+ValueError) .value .reason
├─ InsufficientFundsError    .requested .available
│   └─ OverdraftLimitExceededError + .limit
├─ AccountError
│   ├── AccountNotFoundError (+KeyError)
│   ├── DuplicateAccountError
│   └─ AccountClosedError
├─ TransferError
└─ UnknownAccountTypeError (+ValueError)
```

Everything roots at BankError

A caller can write `except BankError` and be certain. A library that raises bare ValueError from six places cannot be caught reliably.

Overdraft subclasses Insufficient

Code that only cares about 'not enough money' keeps working; code that cares about the overdraft catches the narrower type. Design for the except clauses people will write.

Exceptions carry data

exc.available is what lets the caller tell the customer how much they CAN withdraw. A string message cannot be acted on.

Design your exception hierarchy around how it will be caught, not around how it will be raised.

The ten that cost the most marks

✗ **float anywhere in a money path**

✓ Decimal, fed through str

✗ **`__eq__` without `__hash__`**

✓ define both, keep them consistent

✗ **Public balance attribute**

✓ private state + read-only property

✗ **Overriding `withdraw` in subclasses**

✓ one abstract property, one algorithm

✗ **raise without `from exc`**

✓ chain it; keep the cause

✗ **raise `TypeError` in a dunder**

✓ return `NotImplemented`

✗ **Forgetting `super().__init__()`**

✓ cooperative super, `*args`, `**kwargs`

✗ **Returning the internal list**

✓ return `tuple(self._ledger)`

✗ **`except:` or `except Exception:`**

✓ catch the specific error you handle

✗ **`if/elif` chain in the factory**

✓ a registry dict

None of this was about banking

Every pattern in the exercise has a name and a home in systems you will work on.

Money	Value object	Any exact quantity: currency, physical units, coordinates, versions.
Transaction	Event record	Event sourcing, audit logs, message queues — immutable facts with ids.
Mixins	Cross-cutting concern	Serialisation, logging, caching — capabilities orthogonal to your hierarchy.
withdrawable_balance	Template method	One algorithm, one varying step. Frameworks are built out of this.
Strategies	Strategy + Composite	Pricing rules, validators, retry policies — anything configured at runtime.
Registry factory	Open/closed	Plugin systems, dependency injection, feature flags.
Compensating transfer	Saga pattern	Distributed transactions across microservices. Same problem, more machines.

What to take away

Correctness is the floor, not the ceiling

Every anti-pattern in this deck passes at least one test. Design is what the tests cannot check for you.

Put each rule in exactly one place

One `_apply`, one `_debit`, one `_validate`. When the rule changes, you change one line.

Let the type carry the meaning

An enum instead of a string, a positive amount plus a direction, an exception that carries the numbers.

Catch what you can handle, chain what you cannot

Narrow `excepts` and ``from exc`` are the difference between a debuggable system and a haunted one.

Re-run the suite tonight on your own code. Fix one thing from the ten-mistakes slide, and read the diff.